Instrucciones Generales

- **Archivo base:** Debes usar un archivo `index.html` proporcionado por tu profesor.
  image_c2dca0.png
- **Estructura de carpetas:** Cada ejercicio debe realizarse en su propia carpeta (`~/docker-imatges/ex1`, `ex2`, etc.). Cada una debe contener su propio `Dockerfile` y una copia del `index.html`.
  image_c2dca0.png
- **Nombres y puertos:** Tienes que respetar estrictamente la imagen base indicada, el nombre del contenedor y el puerto asignado en cada ejercicio.
- **Nombre de la imagen:** Las imágenes deben llevar tu nombre de usuario de Docker Hub delante (por ejemplo, `mgarcia/nombre-imagen:tag`).
  image_c2dca0.png
- **Comprobación:** Debes verificar que cada contenedor funciona entrando desde el navegador o usando el comando `curl http://localhost:PORT`.
  image_c2dca0.png

📁 **Información Útil (Tabla de referencia)**

- **Rutas donde se guarda el `index.html` según el servidor:**
  - **NGINX:** `/usr/share/nginx/html/`
    image_c2dca0.png
  - **Apache (`httpd`):** `/usr/local/apache2/htdocs/`
    image_c2dca0.png
- **Gestión de paquetes según el tipo de imagen:**
  - Si la imagen usa **Alpine** (ej. `alpine`, `nginx:alpine`): Se usa el comando `apk add` (sin necesidad de hacer un `update` previo).
    image_c2dca0.png
  - Si la imagen usa **Debian** (la gran mayoría de imágenes estándar): Se debe usar `apt-get update` antes de instalar con `apt-get install -y`.

🏋️ **Desglose de los Ejercicios**

Exercici 1

- **Objetivo:** Crear una imagen basada en `nginx:1.27-alpine`.
  image_c2dca0.png
- **Tareas:**

1. Instalar el paquete `nano` dentro de la imagen.
   image_c2dca0.png
2. Copiar el `index.html` a la ruta de Nginx.
   image_c2dca0.png
3. Nombrar la imagen como `<usuari>/web-nginx:1.0`.
   image_c2dca0.png
4. Arrancar un contenedor llamado `web1` en el **puerto 8081** de tu ordenador.
   image_c2dca0.png
5. Comprobar que `nano` está instalado ejecutando: `docker exec web1 nano --version`.
   image_c2dca0.png

Exercici 2

- **Objetivo:** Crear una imagen basada en `httpd:2.4.62` (servidor Apache, que utiliza Debian por debajo).
- **Tareas:**
  1. Actualizar e instalar el paquete `curl` (recuerda usar `apt-get update` por ser Debian).
  2. Copiar el `index.html` a la ruta de Apache (`/usr/local/apache2/htdocs/`).
  3. Nombrar la imagen como `<usuari>/web-apache:1.0`.
  4. Arrancar un contenedor llamado `web2` en el **puerto 8082** de tu ordenador.
  5. Comprobar que `curl` está instalado ejecutando: `docker exec web2 curl --version`.

Exercici 3

- **Objetivo:** Crear una imagen basada en `nginx:1.26` (versión basada en Debian).
- **Tareas:**
  1. Instalar el paquete `nano` (usando `apt-get update` e `install`).
  2. Copiar el `index.html` a la ruta de Nginx.
  3. Nombrar la imagen como `<usuari>/web-nginx:2.0`.
  4. Arrancar un contenedor llamado `web3` en el **puerto 8083**.
  5. Comprobar que `nano` funciona con: `docker exec web3 nano --version`.

Exercici 4

- **Objetivo:** Crear una imagen basada en `httpd:2.4-alpine` (versión ligera de Apache).
- **Tareas:**
    1. Instalar el paquete `curl` (usando `apk add`, ya que es Alpine).
    2. Copiar el `index.html` a la ruta de Apache.
    3. Nombrar la imagen como `<usuari>/web-apache:2.0`.
    4. Arrancar un contenedor llamado `web4` en el **puerto 8084**.
    5. Comprobar que `curl` funciona con: `docker exec web4 curl --version`.

Exercici 5

- **Objetivo:** Análisis de tamaños.
- **Tareas:**
    - Ejecuta el comando `docker images` para ver el tamaño de las 4 imágenes que has creado.
    - Debes averiguar **cuál es la más pequeña y explicar por qué** crees que lo es (la pista principal es la diferencia entre usar **Alpine** frente a **Debian** como sistema base).
    - Escribe la respuesta en un archivo llamado `README.md` dentro de tu repositorio.

Exercici 6 — Publicación en Docker Hub

- **Objetivo:** Subir tu trabajo a la nube de Docker.
- **Tareas:**
    - Inicia sesión en la terminal con `docker login`.
    - Sube las cuatro imágenes usando `docker push <usuari>/<nombre-imagen>:<tag>`.
    - Entra a la web de Docker Hub para confirmar que las 4 imágenes aparecen publicadas correctamente con sus respectivas etiquetas (*tags*).

Exercici 7 — Entrega en GitHub

- **Objetivo:** Entregar el código fuente al profesor.
- **Tareas:**
    - Crea un repositorio en GitHub llamado `practica-dockerfile-1`.
    - Sube toda la carpeta `~/docker-imatges` asegurándote de que contenga:
        - Las carpetas de `ex1` a `ex4` (cada una con su `Dockerfile` y su `index.html`).

        - El fichero `README.md` con tu nombre y la respuesta del ejercicio 5.
    - Rellena al final tu usuario de Docker Hub y la URL del repositorio de GitHub para entregarlo.

Aquí tienes los Dockerfile y los comandos resueltos paso a paso para cada uno de los ejercicios de la práctica de Docker.
Recuerda que antes de empezar en cada carpeta debes asegurarte de tener una copia del archivo index.html proporcionado por tu profesor. Sustituye <usuari> por tu nombre de usuario real de Docker Hub.

Exercici 1

- **Carpeta:** ~/docker-imatges/ex1
- **Imagen base:** nginx:1.27-alpine
- **Puerto:** 8081

1. Contenido del Dockerfile:

Dockerfile

```
None
FROM nginx:1.27-alpine
RUN apk add --no-cache nano
COPY index.html /usr/share/nginx/html/index.html
```

2. Comandos a ejecutar en la terminal:

Bash

```
None
docker build -t <usuari>/web-nginx:1.0 .
docker run -d --name web1 -p 8081:80 <usuari>/web-nginx:1.0
curl http://localhost:8081
docker exec web1 nano --version
```

Exercici 2

- **Carpeta:** ~/docker-imatges/ex2
- **Imagen base:** httpd:2.4.62 (Usa Debian por debajo)
- **Puerto:** 8082

1. Contenido del Dockerfile:

Dockerfile

```
None
FROM httpd:2.4.62
RUN apt-get update && apt-get install -y curl
COPY index.html /usr/local/apache2/htdocs/index.html
```

2. Comandos a ejecutar en la terminal:

Bash

```
None
docker build -t <usuari>/web-apache:1.0 .
docker run -d --name web2 -p 8082:80
<usuari>/web-apache:1.0
curl http://localhost:8082
docker exec web2 curl --version
```

Exercici 3

- **Carpeta:** ~/docker-imatges/ex3
- **Imagen base:** nginx:1.26 (Usa Debian por debajo)
- **Puerto:** 8083

1. Contenido del Dockerfile:

Dockerfile

```
None
FROM nginx:1.26
RUN apt-get update && apt-get install -y nano
COPY index.html /usr/share/nginx/html/index.html
```

2. Comandos a ejecutar en la terminal:

Bash

```
None
docker build -t <usuari>/web-nginx:2.0 .
docker run -d --name web3 -p 8083:80 <usuari>/web-nginx:2.0
curl http://localhost:8083
docker exec web3 nano --version
```

Exercici 4

- **Carpeta:** ~/docker-imatges/ex4
- **Imagen base:** httpd:2.4-alpine (Usa Alpine)

- **Puerto:** 8084

1. Contenido del Dockerfile:

Dockerfile

```
None
FROM httpd:2.4-alpine
RUN apk add --no-cache curl
COPY index.html /usr/local/apache2/htdocs/index.html
```

2. Comandos a ejecutar en la terminal:

Bash

```
None
docker build -t <usuari>/web-apache:2.0 .
docker run -d --name web4 -p 8084:80
<usuari>/web-apache:2.0
curl http://localhost:8084
docker exec web4 curl --version
```

Exercici 5 (Resolución para el archivo README.md)

Ejecuta el comando para ver las imágenes:

Bash

```
None
docker images
```

- **¿Cuál es la más pequeña?** Las imágenes basadas en **Alpine** (web-nginx:1.0 y web-apache:2.0) serán notablemente más pequeñas que las basadas en Debian (web-apache:1.0 y web-nginx:2.0).
- **¿Por qué?** Alpine Linux es una distribución de Linux extremadamente minimalista (pesa apenas unos pocos megabytes), diseñada específicamente para contenedores, mientras que las imágenes estándar basadas en Debian incluyen muchas más librerías y herramientas predeterminadas que aumentan el peso final de la imagen.

Exercici 6 (Subida a Docker Hub)

Ejecuta los siguientes comandos en tu terminal:

Bash

None

```
docker login
docker push <usuari>/web-nginx:1.0
docker push <usuari>/web-apache:1.0
docker push <usuari>/web-nginx:2.0
docker push <usuari>/web-apache:2.0
```

Exercici 7 (Estructura para GitHub)

Crea el repositorio practica-dockerfile-1 en GitHub y sube la siguiente estructura de carpetas y archivos:

Plaintext

None

```
~/docker-imatges/
├── ex1/
│   ├── Dockerfile
│   └── index.html
├── ex2/
│   ├── Dockerfile
│   └── index.html
├── ex3/
│   ├── Dockerfile
│   └── index.html
├── ex4/
│   ├── Dockerfile
│   └── index.html
└── README.md
```